

Domain-Driven Design · Symfony 8.1 · PHP 8.4

Проект «Catalog»

Как устроен бэкенд каталога товаров и зачем нужен каждый его слой.
Объяснение для тех, кто не знаком с DDD и Symfony.

Учебный разбор архитектуры · бэкенд `higload_app`

Один ограниченный контекст (bounded context): **Catalog**

Товары, категории, фильтрация каталога и привязка категории к товару

Содержание

1. Что это за проект и что он умеет
2. Что такое Symfony (и зачем фреймворк)
3. Что такое DDD и какую проблему он решает
4. Четыре слоя архитектуры
5. Главные понятия DDD на примере этого проекта
6. CQRS: команды и запросы
7. Механизмы Symfony, которые всё связывают
8. Сквозной сценарий №1: создание товара (POST)
9. Сквозной сценарий №2: чтение и список товаров (GET)
10. Сквозной сценарий №3: категории и привязка товара к категории
11. HTTP API Catalog
12. Карта файлов проекта
13. Зачем всё это: выгоды подхода
14. Краткий словарь терминов

1. Что это за проект и что он умеет

Это **серверная часть (бэкенд)** интернет-магазина — а точнее, его ограниченный контекст, отвечающий за **каталог товаров**. Сейчас Catalog умеет работать не только с товарами, но и с категориями: создавать категории, создавать товары с необязательной категорией, получать товары списком, фильтровать их по категории и менять категорию уже существующего товара.

Возможность	HTTP-эндпоинт	Что делает
Создать товар	<code>POST /api/catalog/products</code>	Сохраняет товар, цену, остаток и необязательную ссылку на категорию.
Получить список товаров	<code>GET /api/catalog/products</code>	Возвращает товары; поддерживает <code>q</code> , <code>categoryId</code> , <code>limit</code> , <code>offset</code> .
Получить товар по id	<code>GET /api/catalog/products/{id}</code>	Возвращает одну карточку товара вместе с категорией, если она задана.
Привязать категорию к товару	<code>PATCH /api/catalog/products/{id}/category</code>	Меняет категорию существующего товара или снимает её, если передан <code>null</code> .
Создать категорию	<code>POST /api/catalog/categories</code>	Сохраняет категорию с уникальным <code>slug</code> .
Получить список категорий	<code>GET /api/catalog/categories</code>	Возвращает категории для витрины и админ-панели.
Получить категорию по id	<code>GET /api/catalog/categories/{id}</code>	Возвращает одну категорию.

Аналогия. Представьте склад с папками. Товар — это карточка позиции, категория — раздел каталога. Создание товара — «оформить карточку», создание категории — «завести раздел», а привязка категории — «переложить карточку в нужный раздел, не меняя сам товар».

Технологии: язык **PHP 8.4**, фреймворк **Symfony 8.1**, база данных **PostgreSQL** (через библиотеку Doctrine), быстрый кэш **Redis**. Дальше разберём, что значит каждое слово и зачем оно тут.

2. Что такое Symfony (и зачем нужен фреймворк)

Фреймворк — это готовый «каркас» приложения. Без него вам пришлось бы самим писать кучу скучного кода: как принять HTTP-запрос, как разобрать адрес (URL), как достать данные из базы, как вернуть ответ. Фреймворк всё это уже умеет — вы пишете только свою бизнес-логику.

Symfony — один из двух самых популярных PHP-фреймворков. Из того, что он даёт нашему проекту, важны четыре вещи:

Механизм Symfony	Что делает простыми словами
Роутинг (Routing)	Сопоставляет адрес запроса (<code>/api/catalog/products</code>) с конкретным методом в коде, который его обработает.
Контейнер / DI	«Сборщик» объектов. Сам создаёт нужные классы и передаёт их друг другу. Подробнее — в разделе 7.
Messenger	«Почта» внутри приложения: пересылает сообщения (команды, запросы, события) от отправителя к обработчику.
Doctrine	Переводчик между объектами PHP и таблицами базы данных (ORM). Вы работаете с объектом <code>Product</code> , а он сам пишет SQL.

Главная мысль

Symfony — это про *технику* (HTTP, база, очереди). DDD (следующий раздел) — это про то, как *организовать бизнес-логику*, чтобы она не превратилась в кашу. Эти две вещи дополняют друг друга.

3. Что такое DDD и какую проблему он решает

DDD (Domain-Driven Design, «проектирование на основе предметной области») — это подход к организации кода. Главная идея простая: **в центре приложения должны быть бизнес-правила, а не база данных и не фреймворк.**

Проблема, которую он решает

В типичном «быстром» приложении бизнес-правила размазаны повсюду: немного в контроллере, немного в SQL-запросе, немного в шаблоне. Пока проект маленький — нормально. Когда он растёт, такой код становится трудно менять: одно изменение ломает три других места, а понять «где же правило про отрицательный остаток» невозможно.

Решение DDD

DDD говорит: выдели **«предметную область» (Domain)** — сердце с чистыми бизнес-правилами — и не пускай туда технические детали. База данных, HTTP, кэш — это «*périphérie*», они снаружи и могут меняться, не задевая ядро.

Аналогия с рестораном. Domain — это рецепты и стандарты блюд (что считается правильным борщом). Infrastructure — это плита, холодильник, поставщик продуктов (их можно заменить, рецепт не изменится). UI — это официант, принимающий заказ. DDD требует: рецепт не должен зависеть от того, какой марки у вас холодильник.

Ключевой принцип: зависимости направлены внутрь

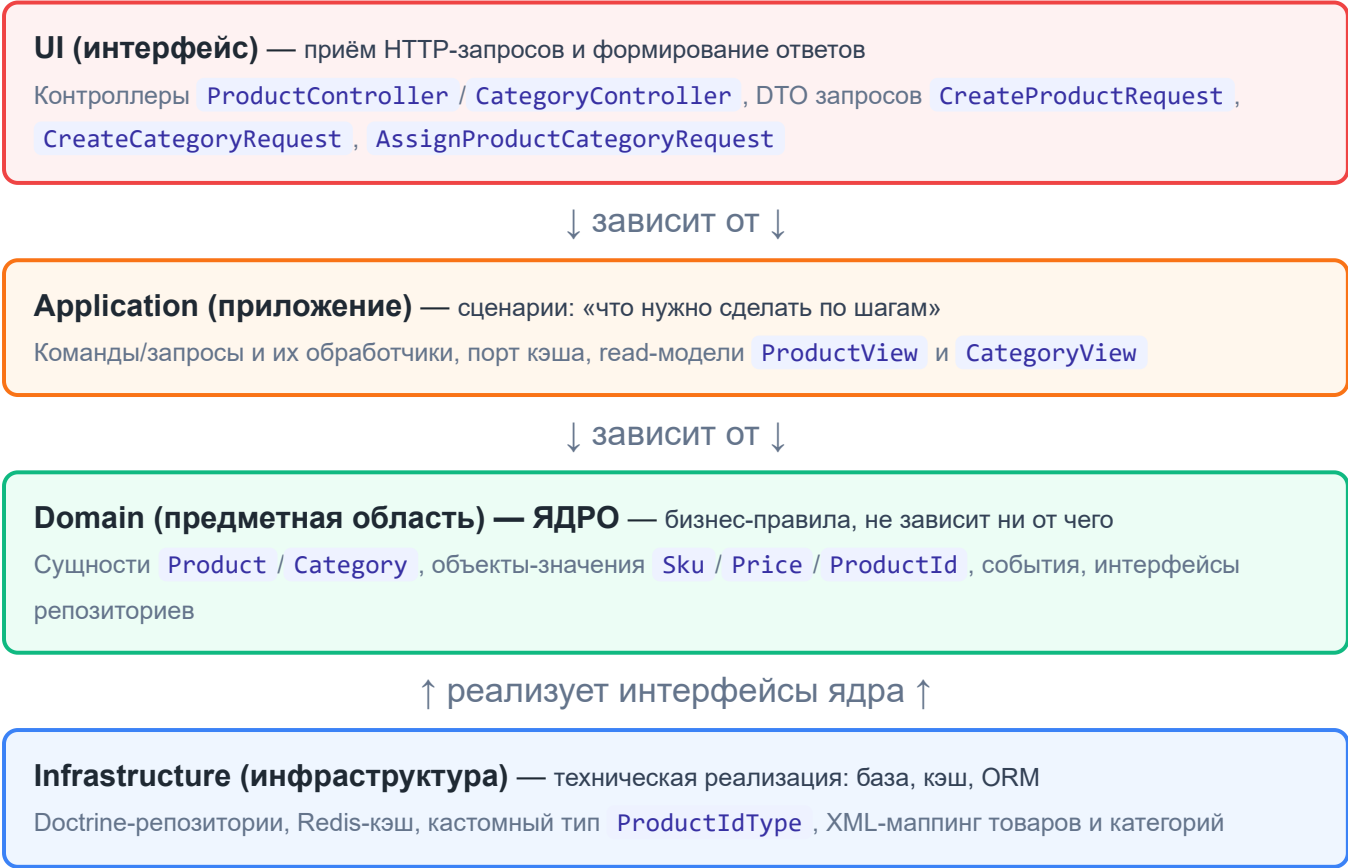
Внешние слои (UI, Infrastructure) знают о ядре (Domain) и зависят от него. Но

ядро не знает о них ничего

. **Product** не подозревает, что существует PostgreSQL или HTTP. Благодаря этому ядро легко тестировать и оно не ломается при смене технологий.

4. Четыре слоя архитектуры

Проект разложен по папкам, и каждая папка — это «слой» с чёткой ответственностью. Посмотрите на путь к файлам: `src/Catalog/Domain/...` , `.../Application/...` , `.../Infrastructure/...` , `.../UI/...` .



5. Главные понятия DDD на примере этого проекта

5.1. Ограниченный контекст (Bounded Context)

Большую систему делят на самостоятельные части — контексты. У нас один контекст — **Catalog** (каталог). В большом магазине были бы ещё «Корзина», «Оплата», «Доставка». Каждый контекст — свой мир со своими правилами и своим словарём. Это видно по пути `src/Catalog/...` — всё лежит внутри одного контекста.

5.2. Сущность и Агрегат (Entity / Aggregate): `Product`

Сущность — объект, у которого есть уникальный идентификатор и своя «жизнь» во времени (его можно менять, но это всё тот же объект). Наш `Product` — это сущность и одновременно **агрегат**: главный объект, который отвечает за свою целостность. У товара есть артикул, название, описание, цена, остаток, даты создания/обновления и необязательная ссылка на `Category`.

Важно: правила живут *внутри* объекта. Нельзя положить отрицательный остаток — объект сам это запретит. Изменение категории тоже проходит через метод домена, а не через прямую запись поля из контроллера:

```
public function updateStock(int $newStock): void
{
    if ($newStock < 0) throw new \DomainException('Stock cannot be negative');
    $this->stock = $newStock;
    $this->updatedAt = new \DateTimeImmutable();
    $this->recordUpdated(); // запомнить, что товар изменился
}
```

```
public function assignCategory(?Category $category): void
{
    $this->category = $category; // null означает «без категории»
    $this->updatedAt = new \DateTimeImmutable();
    $this->recordUpdated();
}
```

Почему это хорошо

Невозможно «случайно» создать неправильный товар из другого места программы. Правило в одном месте — в самом объекте. Это и есть «защита инвариантов».

5.3. Сущность `Category` : раздел каталога

`Category` — отдельная сущность каталога. У неё свой UUID, название, `slug`, описание и даты. Товар может ссылаться на категорию, но категория живёт самостоятельно: её можно создать до товара, а потом назначить разным товарам.

```
class Category
{
    public const SLUG_PATTERN = '/^[a-z0-9]+(?:-[a-z0-9]+)*$/';

    private string $id;
    private string $name;
    private string $slug;

    private static function normalizeSlug(string $slug): string
    {
        $slug = strtolower(trim($slug));
        if (!preg_match(self::SLUG_PATTERN, $slug)) {
            throw new \InvalidArgumentException('Invalid category slug');
        }
        return $slug;
    }
}
```

Почему категория не Value Object

Категория имеет собственную идентичность и жизненный цикл: её можно создать, переименовать, найти по id или slug, а товары только ссылаются на неё. Поэтому это сущность, а не объект-значение.

5.4. Объект-значение (Value Object): `Sku`, `Price`, `ProductId`

Объект-значение — это маленький неизменяемый объект, который описывается только своим значением и не имеет собственного «id». Деньги, артикул, идентификатор — отличные кандидаты.

Зачем заворачивать цену в класс `Price`, а не хранить просто число? Чтобы **правила и смысл** были рядом со значением:

```
final class Price
{
    public function __construct(private int $amount, private string $currency = 'KZT')
    {
        if ($amount < 0) throw new \InvalidArgumentException('Price cannot be negative');
    }
    // два объекта-значения равны, если равны их значения
    public function equals(self $other): bool
    {
        return $this->amount === $other->amount && $this->currency === $other->currency;
    }
}
```

Аналогия. Купюра в 100 тенге равноценна любой другой купюре в 100 тенге — нам не важен её серийный номер. Это «объект-значение». А вот сотрудник Иван — это «сущность»: даже два полных тёзки — разные люди.

Артикул `Sku` хранит правило формата прямо в себе, в виде константы (её же переиспользует слой UI для проверки ввода — правило не дублируется):

```
final class Sku
{
    public const PATTERN = '/^[A-Z0-9]{8,20}$/'; // 8-20 заглавных букв и цифр
}
```

5.5. Доменные события (Domain Events): `ProductCreatedEvent`, `ProductUpdatedEvent`

Доменное событие — это факт «что-то важное произошло в бизнесе»: *товар создан*, *товар изменён*. Агрегат сам «записывает» такие факты, когда меняется. Позже их можно разослать другим частям системы (поиск, аналитика, уведомления) — и им не нужно для этого лезть внутрь нашего кода.

```
// внутри Product: накапливаем события, потом их «забирают»
public function pullDomainEvents(): array
{
    $events = $this->domainEvents;
    $this->domainEvents = []; // очищаем, чтобы не разослать дважды
    return $events;
}
```

Тонкость DDD, которую мы исправили

Раньше событие собиралось в обработчике (снаружи объекта). Это неправильно: «знание о том, что товар создан» — принадлежит самому товару. Теперь `Product` сам записывает событие в конструкторе, а обработчик лишь забирает и рассылает.

5.6. Репозиторий (Repository) и «порт»

Репозиторий — это «хранилище объектов»: умеет сохранить товар/категорию и найти их по id. Хитрость DDD: ядро объявляет *только интерфейс* (что должно уметь хранилище), а *как именно* — пишет Infrastructure.

```
// Domain: контракт (порт). Никакого SQL — только намерение.
interface ProductRepositoryInterface
{
    public function save(Product $product): void;
    public function findById(ProductId $id): ?Product;
    public function findByCriteria(array $filters, int $limit, int $offset): array;
}

interface CategoryRepositoryInterface
{
    public function save(Category $category): void;
    public function findById(string $id): ?Category;
}
```

```
public function findBySlug(string $slug): ?Category;
}
```

```
// Infrastructure: реализация через Doctrine (здесь уже база)
class ProductRepositoryDoctrine implements ProductRepositoryInterface
{
    public function save(Product $product): void
    {
        $this->em->persist($product);
        $this->em->flush(); // здесь Doctrine пишет SQL INSERT
    }
}
```

Зачем такая «двойственность»

Это называется «инверсия зависимостей». Завтра вы смените PostgreSQL на MongoDB — поменяете только класс в Infrastructure, а всё ядро и сценарии останутся нетронутыми, потому что они зависят от *интерфейса*, а не от базы.

6. CQRS: команды и запросы

CQRS (Command Query Responsibility Segregation) — «разделение ответственности команд и запросов». Идея в одном предложении: **операции, которые меняют данные, и операции, которые только читают данные, — это разные миры, и обрабатывать их надо по-разному.**

	Команда (Command)	Запрос (Query)
Цель	Изменить состояние («создай товар», «создай категорию», «назначь категорию товару»)	Прочитать состояние («дай товар», «дай список товаров», «дай категории»)
Возвращает	Ничего, подтверждение или read-модель изменённого объекта	Данные для показа
В проекте	CreateProductCommand , CreateCategoryCommand , AssignProductCategoryCommand	GetProductQuery , ListProductsQuery , GetCategoryQuery , ListCategoriesQuery
Идёт через шину	command.bus	query.bus

Команда — это просто «конверт с данными» о намерении. Сама она ничего не делает; её исполняет **обработчик (handler)**:

```
class CreateProductCommand // просто данные намерения
{
    public function __construct(
        public readonly Sku $sku,
        public readonly string $name,
        public readonly Price $price,
        public readonly int $stock,
        public readonly ?string $description,
        public readonly ?string $categoryId = null
    ) {}
}
```

Для изменения категории у уже существующего товара используется отдельная команда. Это важно: создание товара и назначение категории — разные сценарии, и у каждого есть свой обработчик.

```
final class AssignProductCategoryCommand
{
    public function __construct(
        public readonly ProductId $productId,
        public readonly ?string $categoryId
    ) {}
}
```

```
) {}  
}
```

Зачем разделять

Чтение часто можно ускорить (кэш, упрощённые модели данных), а запись — требует строгих проверок. Разделив их, мы оптимизируем каждую сторону отдельно. В нашем проекте чтение использует кэш Redis и лёгкую модель `ProductView`, а запись идёт через полноценный агрегат `Product` с проверками.

7. Механизмы Symfony, которые всё связывают

7.1. Контейнер и внедрение зависимостей (Dependency Injection)

Обработчикам вроде `CreateProductHandler`, `CreateCategoryHandler` и `AssignProductCategoryHandler` нужны репозитории, кэш и шина событий. Они их **не создают сами** — они просто заявляют в конструкторе «мне нужно вот это», а Symfony-контейнер при запуске сам создаёт и «вкладывает» нужные объекты. Это и есть **внедрение зависимостей**.

Аналогия. Вы не строите автомобиль, чтобы доехать до работы, — вам его «подают». Классу подают всё, что ему нужно, готовым.

В файле `config/services.yaml` мы говорим контейнеру, какую *реализацию* подставлять под *интерфейс*:

```
# «Когда кому-то нужен интерфейс репозитория — дай Doctrine-реализацию»
App\Catalog\Domain\Repository\ProductRepositoryInterface:
    '@App\Catalog\Infrastructure\Doctrine\ProductRepositoryDoctrine'
```

```
App\Catalog\Domain\Repository\CategoryRepositoryInterface:
    '@App\Catalog\Infrastructure\Doctrine\CategoryRepositoryDoctrine'

App\Catalog\Application\Port\ProductCacheInterface:
    '@App\Catalog\Infrastructure\Cache\ProductCacheRedis'
```

7.2. Messenger и три шины

Шина (bus) — это «почтовая служба»: вы бросаете в неё сообщение, она находит нужный обработчик и доставляет. У нас три шины под три типа сообщений (`config/packages/messenger.yaml`):

Шина	Для чего
<code>command.bus</code>	Команды на изменение: создать товар, создать категорию, назначить категорию товару
<code>query.bus</code>	Запросы на чтение: получить товар, список товаров, категорию, список категорий
<code>event.bus</code>	Доменные события (товар создан/изменён) — рассылаются асинхронно

7.3. Doctrine ORM: объекты ↔ таблицы

ORM (Object-Relational Mapping) избавляет от ручного написания SQL. Вы работаете с объектами `Product` и `Category`, а Doctrine знает, в какие таблицы и колонки их положить. «Карта» соответствия задана XML-файлами в `Infrastructure/Doctrine/Mapping`.

Объекты-значения `Sku` и `Price` хранятся не в отдельных таблицах, а «встраиваются» в строку товара (это называется *embeddable*): цена превращается в две колонки — `price_amount` и `price_currency`.

Связь товара с категорией хранится как nullable many-to-one: в таблице `catalog_products` есть колонка `category_id`, которая ссылается на `catalog_categories.id`. Nullable означает, что товар может существовать без категории. В XML-маппинге это выглядит так:

```
<many-to-one field="category" target-entity="App\Catalog\Domain\Entity\Category">
  <join-column name="category_id" referenced-column-name="id"
    nullable="true" on-delete="SET NULL"/>
</many-to-one>
```

А чтобы наш `ProductId` (объект) корректно превращался в строку UUID для базы и обратно, написан **кастомный тип** `ProductIdType`:

```
// объект ProductId → строка для базы
public function convertToDatabaseValue($value, $platform): ?string
{
    return $value instanceof ProductId ? $value->toString() : null;
}
```

7.4. Кэш Redis за «портом»

Кэш — быстрая память для часто запрашиваемых данных, чтобы не дёргать базу каждый раз. **Redis** — популярное хранилище для этого. Чтобы слой Application не зависел от Redis напрямую, мы спрятали кэш за интерфейс-порт `ProductCacheInterface`; реализация `ProductCacheRedis` живёт в Infrastructure. Кэш хранит лёгкую read-модель `ProductView` на 5 минут.

Когда категория существующего товара меняется через `AssignProductCategoryHandler`, кэш конкретного товара инвалидируется: следующий `GET /api/catalog/products/{id}` уже вернёт актуальную категорию.

7.5. Контроллер и проверка входных данных

Контроллер — «дверь» приложения. Symfony сам превращает входной JSON в объект запроса и проверяет его (атрибут `#[MapRequestPayload]`). Если данные кривые — клиент получит ошибку ещё до того, как они дойдут до бизнес-логики.

```
#[Route('/api/catalog/products', methods: ['POST'])]
public function create(#[MapRequestPayload] CreateProductRequest $request): JsonResponse
```

Для разных действий используются разные DTO: `CreateProductRequest` проверяет товар, `CreateCategoryRequest` проверяет название и `slug` категории, а `AssignProductCategoryRequest` проверяет, что `categoryId` либо UUID, либо `null`.

8. Сквозной сценарий №1: создание товара (POST)

Проследим, что происходит, когда приходит `POST /api/catalog/products` с JSON товара. В запросе можно передать `categoryId`, но это поле необязательное: товар может быть создан без категории.

- 1 **UI.** Symfony по адресу находит метод `ProductController::create`. JSON автоматически превращается в `CreateProductRequest` и проверяется (артикул, длина имени, цена > 0 , остаток ≥ 0 , `categoryId` — UUID или `null`).
- 2 **UI → Application.** Контроллер собирает из проверенных данных `CreateProductCommand` и бросает её в `command.bus`.
- 3 **Application.** Если `categoryId` передан, `CreateProductHandler` ищет категорию через `CategoryRepositoryInterface`. Если категории нет — возвращается ошибка **400 Bad Request**.
- 4 **Domain.** Обработчик создаёт **новый агрегат** `new Product(..., $category)`. Товар проверяет свои правила и записывает событие «товар создан».
- 5 **Application → Infrastructure.** Обработчик вызывает `repository->save($product)`. Doctrine-репозиторий пишет строку в PostgreSQL.
- 6 **События.** Обработчик забирает у товара события (`pullDomainEvents()`) и рассылает их в `event.bus`.
- 7 **Ответ.** Контроллер возвращает JSON `{"status": "Product created"}` и HTTP-код **201 Created**.

```
// Фрагмент CreateProductHandler
$category = null;
if ($command->categoryId !== null) {
    $category = $this->categoryRepository->findById($command->categoryId);
    if (!$category) {
        throw new \DomainException('Category not found');
    }
}

$product = new Product($command->sku, $command->name, $command->price,
    $command->stock, $command->description, $category);
$this->repository->save($product);
```

9. Сквозной сценарий №2: чтение и список товаров (GET)

В Catalog есть два сценария чтения товаров: получить один товар по id и получить список товаров. Оба возвращают read-модель `ProductView`, где категория вложена как объект `CategoryView` или равна `null`.

9.1. Один товар: `GET /api/catalog/products/{id}`

- 1 **UI.** `ProductController::get` превращает строку `{id}` в объект `ProductId`. Если id невалидный — сразу ответ **400 Bad Request**.
- 2 **UI → Application.** Контроллер бросает `GetProductQuery` в `query.bus`; шина возвращает результат обработчика.
- 3 **Application + кэш.** `GetProductHandler` спрашивает кэш: «есть товар с таким id?». Если есть — отдаёт мгновенно. Если нет — идёт в репозиторий.
- 4 **Ответ.** Если товар найден — `ProductView` превращается в JSON (код **200 OK**). Если нет — **404 Not Found**.

```
public function __invoke(GetProductQuery $query): ?ProductView
{
    return $this->cache->get($query->id, function () use ($query): ?ProductView {
        $product = $this->repository->findById($query->id); // промах кэша → база
        return $product ? ProductView::fromEntity($product) : null;
    });
}
```

9.2. Список товаров: `GET /api/catalog/products`

Список товаров не использует кэш одного товара, потому что это другой read-сценарий: у него есть фильтры, лимит и смещение. Сейчас поддерживаются:

- `q` — поиск по части названия;
- `categoryId` — оставить только товары выбранной категории;
- `limit` и `offset` — простая пагинация.

```
if ($query->categoryId !== null && $query->categoryId !== '') {
    $filters['categoryId'] = $query->categoryId;
}

return array_map(
    static fn ($product): ProductView => ProductView::fromEntity($product),
```



```
$this->repository->findByCriteria($filters, $query->limit, $query->offset)
);
```

Почему отдаём `ProductView`, а не `Product`

Тяжёлый агрегат с правилами незачем (и небезопасно) складывать в кэш и слать наружу. Лёгкая «витрина» `ProductView` содержит только то, что нужно показать, и легко превращается в JSON.

10. Сквозной сценарий №3: категории и привязка товара к категории

Категории добавлены в тот же bounded context **Catalog**, потому что это часть языка каталога: товар может быть «без категории» или находиться в конкретном разделе.

10.1. Создание категории

- 1 **UI.** `CategoryController::create` принимает JSON и валидирует `name`, `slug`, `description`.
- 2 **Application.** `CreateCategoryHandler` проверяет уникальность `slug` через `CategoryRepositoryInterface::findBySlug()`.
- 3 **Domain.** Создаётся `Category`; сама сущность нормализует имя и slug.
- 4 **Ответ.** Контроллер возвращает созданную категорию как `CategoryView` и HTTP-код **201 Created**.

10.2. Привязка категории к уже существующему товару

Для админ-панели нужен сценарий «товар уже создан, теперь выбери ему категорию». Это не должно имитироваться на frontend: выбор должен сохраняться в backend. Поэтому добавлен endpoint `PATCH /api/catalog/products/{id}/category`.

- 1 **UI.** `ProductController::assignCategory` проверяет id товара из URL и payload `{"categoryId": "uuid"}` или `{"categoryId": null}`.
- 2 **Application.** `AssignProductCategoryHandler` находит товар. Если товара нет — **404 Not Found**.
- 3 **Application.** Если `categoryId` не null, обработчик находит категорию. Если категории нет — **404 Not Found**.
- 4 **Domain.** Вызывается `$product->assignCategory($category)`. Товар обновляет `updatedAt` и записывает `ProductUpdatedEvent`.
- 5 **Infrastructure.** Обработчик сохраняет товар, инвалидирует Redis-кэш этого товара и рассылает доменные события.
- 6 **Ответ.** Возвращается обновлённый `ProductView` вместе с актуальной категорией.

```
public function __invoke(AssignProductCategoryCommand $command): ProductView
{
    $product = $this->productRepository->findById($command->productId);
    if (!$product) throw new \DomainException('Product not found');
```

```
$category = null;
if ($command->categoryId !== null && $command->categoryId !== '') {
    $category = $this->categoryRepository->findById($command->categoryId);
    if (!$category) throw new \DomainException('Category not found');
}

$product->assignCategory($category);
$this->productRepository->save($product);
$this->productCache->invalidate($product->getId());

return ProductView::fromEntity($product);
}
```

11. HTTP API Catalog

Итоговая карта HTTP API, которая должна отображаться в `/api/doc.json` :

Метод	Путь	Payload / Query	Ответ
POST	<code>/api/catalog/products</code>	<code>sku , name , priceAmount , currency , stock , description? , categoryId?</code>	201 <code>{"status":"Product created"}</code>
GET	<code>/api/catalog/products</code>	<code>q? , categoryId? , limit? , offset?</code>	Список <code>ProductView</code>
GET	<code>/api/catalog/products/{id}</code>	UUID товара	Один <code>ProductView</code> или 404
PATCH	<code>/api/catalog/products/{id}/category</code>	<code>{"categoryId":"uuid"}</code> или <code>{"categoryId":null}</code>	Обновлённый <code>ProductView</code>
POST	<code>/api/catalog/categories</code>	<code>name , slug , description?</code>	<code>CategoryView</code> и код 201
GET	<code>/api/catalog/categories</code>	<code>limit? , offset?</code>	Список <code>CategoryView</code>
GET	<code>/api/catalog/categories/{id}</code>	UUID категории	Один <code>CategoryView</code> или 404

Связь с frontend
Frontend не создаёт фиктивные поля. Админ-панель берёт категории из `GET /api/catalog/categories` , создаёт товары через `POST /api/catalog/products` , а для уже существующих товаров сохраняет выбор через `PATCH /api/catalog/products/{id}/category` .

12. Карта файлов проекта

Где что лежит и за что отвечает (всё внутри `src/Catalog/`):

Файл	Слой	Назначение
<code>Domain/Entity/Product.php</code>	Domain	Агрегат-товар: правила, события
<code>Domain/Entity/Category.php</code>	Domain	Сущность категории: id, name, slug, описание, даты
<code>Domain/ValueObject/Sku.php</code>	Domain	Артикул + правило формата
<code>Domain/ValueObject/Price.php</code>	Domain	Цена + валюта, запрет отрицательной
<code>Domain/ValueObject/ProductId.php</code>	Domain	Идентификатор товара (UUID)
<code>Domain/Event/ProductCreatedEvent.php</code>	Domain	Факт «товар создан»
<code>Domain/Event/ProductUpdatedEvent.php</code>	Domain	Факт «товар изменён»
<code>Domain/Repository/ProductRepositoryInterface.php</code>	Domain	Контракт хранилища (порт)
<code>Domain/Repository/CategoryRepositoryInterface.php</code>	Domain	Контракт хранилища категорий
<code>Application/Command/CreateProductCommand.php</code>	Application	Намерение «создать товар»
<code>Application/Command/CreateProductHandler.php</code>	Application	Сценарий создания
<code>Application/Command/CreateCategoryCommand.php</code>	Application	Намерение «создать категорию»
<code>Application/Command/CreateCategoryHandler.php</code>	Application	Сценарий создания категории
<code>Application/Command/AssignProductCategoryCommand.php</code>	Application	Намерение «назначить категорию товару»
<code>Application/Command/AssignProductCategoryHandler.php</code>	Application	Сценарий привязки/отвязки категории

<code>Application/Query/GetProductQuery.php</code>	Application	Намерение «получить товар»
<code>Application/Query/GetProductHandler.php</code>	Application	Сценарий чтения (с кэшем)
<code>Application/Query/ListProductsQuery.php</code>	Application	Намерение «получить список товаров» с фильтрами
<code>Application/Query/ListProductsHandler.php</code>	Application	Сценарий списка товаров
<code>Application/Query/GetCategoryQuery.php</code>	Application	Намерение «получить категорию»
<code>Application/Query/GetCategoryHandler.php</code>	Application	Сценарий чтения категории
<code>Application/Query/ListCategoriesQuery.php</code>	Application	Намерение «получить список категорий»
<code>Application/Query/ListCategoriesHandler.php</code>	Application	Сценарий списка категорий
<code>Application/Query/ProductView.php</code>	Application	Лёгкая read-модель для показа
<code>Application/Query/CategoryView.php</code>	Application	Лёгкая read-модель категории
<code>Application/Port/ProductCacheInterface.php</code>	Application	Контракт кэша (порт)
<code>Infrastructure/Doctrine/ProductRepositoryDoctrine.php</code>	Infrastructure	Хранилище через PostgreSQL
<code>Infrastructure/Doctrine/CategoryRepositoryDoctrine.php</code>	Infrastructure	Хранилище категорий через PostgreSQL
<code>Infrastructure/Doctrine/Mapping/Entity.Product.orm.xml</code>	Infrastructure	Маппинг товара, включая <code>category_id</code>
<code>Infrastructure/Doctrine/Mapping/Entity.Category.orm.xml</code>	Infrastructure	Маппинг таблицы <code>catalog_categories</code>
<code>Infrastructure/Doctrine/Type/ProductIdType.php</code>	Infrastructure	Перевод ProductId ↔ строка БД
<code>Infrastructure/Cache/ProductCacheRedis.php</code>	Infrastructure	Кэш через Redis

UI/Http/ProductController.php	UI	HTTP-эндпоинты товаров: create/list/get/assign category
UI/Http/CategoryController.php	UI	HTTP-эндпоинты категорий: create/list/get
UI/Http/Dto/CreateProductRequest.php	UI	Проверка входного JSON товара
UI/Http/Dto/CreateCategoryRequest.php	UI	Проверка входного JSON категории
UI/Http/Dto/AssignProductCategoryRequest.php	UI	Проверка payload для привязки категории

13. Зачем всё это: выгоды подхода

Слоёв и абстракций больше, чем в «простом» коде — и это окупается, когда проект растёт:

- **Бизнес-правила в одном месте.** Хотите узнать все правила про товар — открываете `Product` ; правила категории — `Category` .
- **Легко тестировать.** Ядро не зависит от базы и сети, поэтому правила проверяются быстрыми unit-тестами без запуска PostgreSQL/Redis. В проекте уже есть такие тесты (12 шт.) — они проверяют события агрегата и объекты-значения за доли секунды.
- **Технологии заменяемы.** Сменить базу или кэш = поменять класс в Infrastructure. Ядро и сценарии не трогаются.
- **Чтение и запись масштабируются отдельно** (благодаря CQRS): читаем — из кэша и быстрых моделей, пишем — со строгими проверками.
- **Понятный словарь.** Код говорит на языке бизнеса (Product, Category, Sku, Price, «товар создан», «категория назначена»), а не на языке таблиц.

Честно о минусах

DDD — это «инвестиция». Для крошечного скрипта он избыточен: больше файлов, больше понятий. Он оправдан там, где логика сложная и проект живёт долго (как раз случай растущего магазина — «highload»-приложения, судя по названию проекта).

14. Краткий словарь терминов

Термин	Простыми словами
DDD	Подход: в центре кода — бизнес-правила, а не база/фреймворк
Bounded Context	Самостоятельная часть системы со своим словарём (у нас — Catalog)
Domain	Ядро с бизнес-правилами; ни от чего не зависит
Application	Сценарии «что сделать по шагам»; дирижёр
Infrastructure	Технические детали: база, кэш, ORM
UI	Приём запросов и выдача ответов (HTTP/JSON)
Entity / Сущность	Объект с id и историей изменений (Product, Category)
Category	Раздел каталога; товар может быть привязан к одной категории или быть без категории
Slug	Короткий машинный идентификатор категории в URL/API, например <code>electronics</code>
Nullable relation	Связь, которая может отсутствовать; у товара <code>category_id</code> может быть <code>null</code>
Aggregate / Агрегат	Главная сущность, отвечающая за свою целостность
Value Object	Неизменяемый объект, описываемый только значением (Price, Sku)
Domain Event	Факт «что-то важное произошло» (товар создан)
Repository	Хранилище объектов: сохранить / найти
Port (порт)	Интерфейс в ядре; реализацию даёт Infrastructure
CQRS	Разделение операций записи (Command) и чтения (Query)
DI / Контейнер	Symfony сам создаёт объекты и «подаёт» их зависимостям
Messenger / Bus	Внутренняя «почта»: доставляет сообщения обработчикам
ORM / Doctrine	Переводчик «объекты ↔ таблицы базы данных»
Read model / View	Упрощённая модель для ответа API (<code>ProductView</code> , <code>CategoryView</code>)
DTO	Простой объект-«конверт» для передачи данных (<code>CreateProductRequest</code> , <code>ProductView</code>)

Документ сгенерирован как учебное объяснение архитектуры проекта `higload_app/backend` (контекст Catalog).